UniCesumar
Engenharia de Software

Manutenção de Software

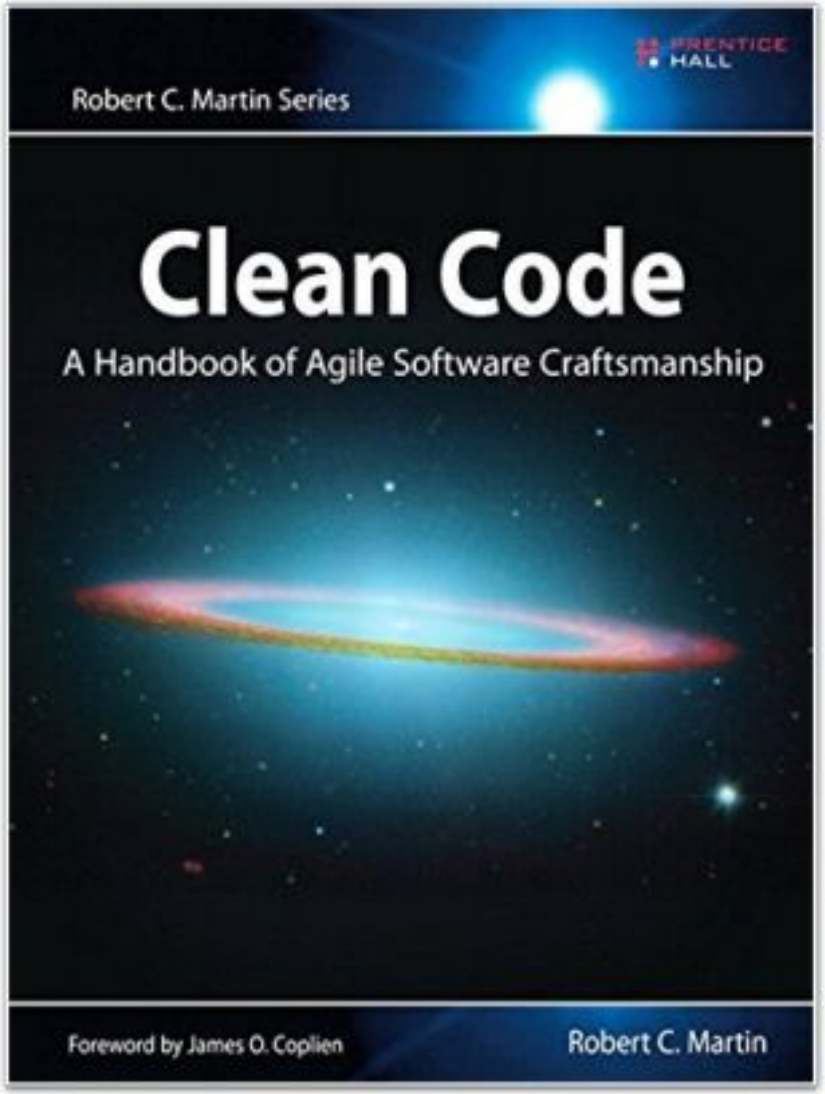
Código Limpo

- Legibilidade

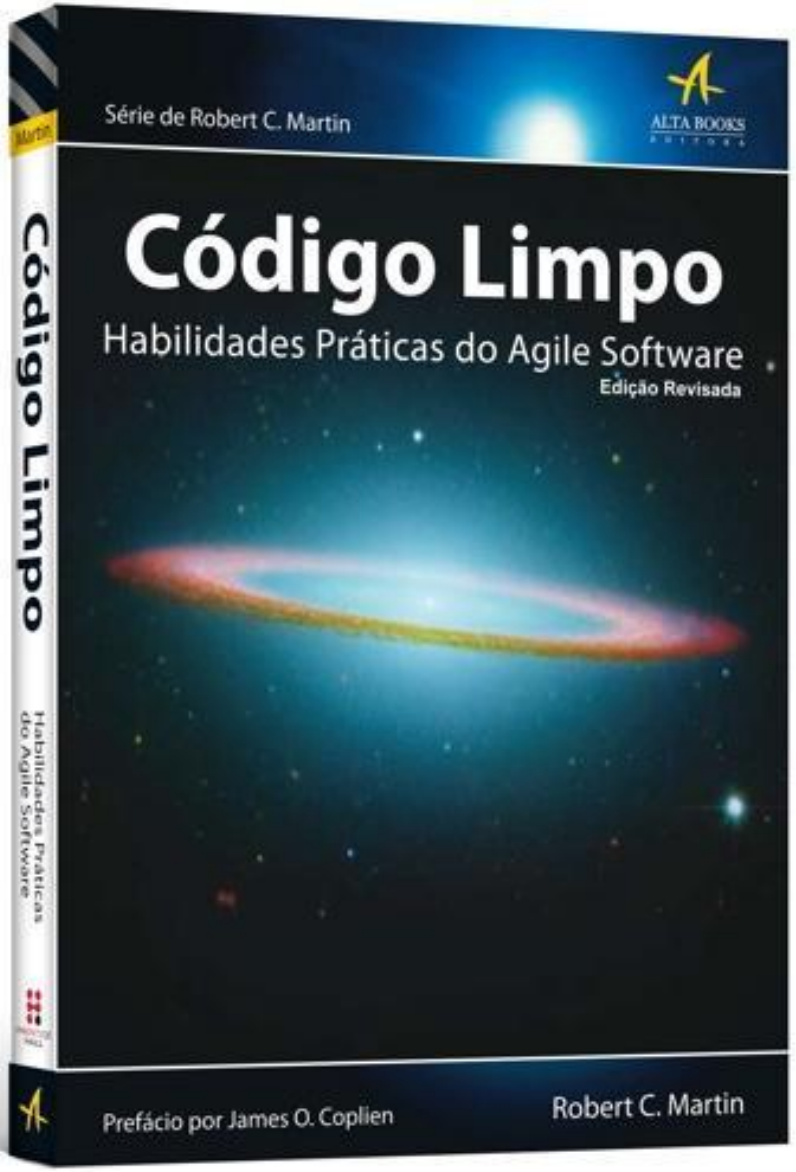
- Comentários

- Formatação

Código Limpo



Código Limpo



Código Limpo

“Você está lendo este livro por duas razões. Primeiro, você é um programador. Segundo, você quer ser um programador melhor. Ótimo. Precisamos de programadores melhores.”

O que é Código Limpo?

“Código limpo é simples e direto. Código limpo se lê como uma prosa bem escrita. Código limpo nunca obscurece a intenção do projetista; ao contrário, é repleto de abstrações e de fluxos de controle claros e diretos. “

Grady Booch – criador da UML

O que é Código Limpo?

"Gosto que meu código seja **elegante** e **eficiente**. A lógica deve ser direta, para dificultar que bugs se escondam; as dependências devem ser mínimas, para facilitar a manutenção; o tratamento de erros deve ser completo, de acordo com uma estratégia bem definida; e o desempenho deve estar próximo do ideal, para não tentar as pessoas a bagunçar o código com otimizações sem critério. Código limpo faz uma coisa bem feita."

Bjarne Stroustrup – criador do C++

O que é Código Limpo?

"Código limpo pode ser lido e aprimorado por um desenvolvedor que não seja seu autor original. Ele possui testes unitários e de aceitação. Tem nomes significativos. Oferece uma forma, em vez de várias, de fazer a mesma coisa. Possui dependências mínimas, explicitamente definidas, e fornece uma API clara e enxuta."

Dave Thomas – criador do Eclipse

Código Limpo

- Legibilidade
- Comentários
- Formatação
- Funções
- Código externo
- Testes de unidade
- Classes

Código Limpo

- **Legibilidade**
- **Comentários**
- **Formatação**
- Funções
- Código externo
- Testes de unidade
- Classes

Código Limpo - Legibilidade

- Nomes são tudo em um software

- Nomeamos variáveis, métodos, argumentos, classes, pacotes, arquivos, diretórios...

Dicas de Legibilidade

- Usar nomes que revelam a intenção

- Fazer distinção entre nomes

- Usar nomes pronunciáveis

- Usar nomes “buscáveis”

O que é Código Limpo?

```
for (int j=0; j<34; j++) {
  s += (t[j]*4)/5;
}
```

O que é Código Limpo?

```
int realDaysPerIdealDay = 4;
const int WORK_DAYS_PER_WEEK = 5;
int sum = 0;
for (int j=0; j < NUMBER_OF_TASKS; j++) {
  int realTaskDays = taskEstimate[j] * realDaysPerIdealDay;
  int realTaskWeeks = (realdays / WORK_DAYS_PER_WEEK);
  sum += realTaskWeeks;
}
```

Dicas de Legibilidade

- Usar nomes que revelam a intenção

  - int d; // tempo em dias

  - int s; // semanas

  - int m; // meses

  - int a; // anos

Dicas de Legibilidade

- Usar nomes que revelam a intenção

  - **int tempoEmDias;**

  - **int diasDesdeACriacao;**

  - **int idadeDoArquivoEmDias;**

Dicas de Legibilidade

- Usar nomes que revelam a intenção

```
public List<int[]> getThem() {
  List<int[]> list1 = new ArrayList<int[]>();
  for (int[] x : theList)
    if (x[0] == 4)
      list1.add(x);
  return list1;
}
```

Dicas de Legibilidade

- Usar nomes que revelam a intenção

```
public List<int[]> getFlaggedCells() {
  List<int[]> flaggedCells = new ArrayList<int[]>();
  for (int[] cell : gameBoard)
   if (cell[STATUS_VALUE] == FLAGGED)
     flaggedCells.add(cell);
   return flaggedCells;
}
```

Dicas de Legibilidade

- Usar nomes que revelam a intenção

```
public List<Cell> getFlaggedCells() {
  List<Cell> flaggedCells = new ArrayList<Cell>();
  for (Cell cell : gameBoard)
    if (cell.isFlagged())
      flaggedCells.add(cell);
  return flaggedCells;
}
```

Dicas de Legibilidade

- Fazer distinção entre nomes

```
public static void copyChars(char a1[], char a2[]) {
  for (int i = 0; i < a1.length; i++) {
    a2[i] = a1[i];
  }
}
```

Dicas de Legibilidade

- Fazer distinção entre nomes

- Copia o que para onde?
- Definir fonte e destino (a1/a2)

```
public static void copyChars(char a1[], char a2[]) {
  for (int i = 0; i < a1.length; i++) {
    a2[i] = a1[i];
  }
}
```

Dicas de Legibilidade

- Usar nomes pronunciáveis

```
class DtaRcrd102 {
  private Date genymdhms;
  private Date modymdhms;
  private final String pszqint = "102";
   /* … */
};
```

Dicas de Legibilidade

- Usar nomes pronunciáveis

```
class Customer {
  private Date generationTimestamp;
  private Date modificationTimestamp;;
  private final String recordId = "102";
  /* … */
};
```

Dicas de Legibilidade

- Usar nomes “buscáveis”

```
for (int j=0; j<34; j++) {
  s += (t[j]*4)/5;
}
```

Dicas de Legibilidade

- Usar nomes “buscáveis”

```
int realDaysPerIdealDay = 4;
const int WORK_DAYS_PER_WEEK = 5;
int sum = 0;
for (int j=0; j < NUMBER_OF_TASKS; j++) {
  int realTaskDays = taskEstimate[j] * realDaysPerIdealDay;
  int realTaskWeeks = (realdays / WORK_DAYS_PER_WEEK);
  sum += realTaskWeeks;
}
```

Código Limpo – Comentários

- Não existe nada mais útil do que um bom comentário

- Não existe nada pior do que um comentário ruim, redundante, desnecessário...

- O propósito dos comentários é compensar a falha de nos expressarmos em código

Código Limpo – Comentários

- Qual sua opinião sobre o código abaixo?

```
// Check to see if the employee is eligible for full benefits
if ((employee.flags & HOURLY_FLAG) &&
    (employee.age > 65))
```

- Tem como melhorar?

Código Limpo – Comentários

- Qual sua opinião sobre o código abaixo?

```
// Check to see if the employee is eligible for full benefits
if ((employee.flags & HOURLY_FLAG) &&
    (employee.age > 65))
```

- Tem como melhorar?

```
if (employee.isEligibleForFullBenefits())
```

Comentários relevantes

- Qual sua opinião sobre o código abaixo?

```
// Returns an instance of the Responder being tested.
protected abstract Responder responderInstance();
```

- Tem como melhorar?

Comentários relevantes

- Qual sua opinião sobre o código abaixo?

```
// Returns an instance of the Responder being tested.
protected abstract Responder responderInstance();
```

- Tem como melhorar?

responderBeingTested( )

Comentários relevantes

- Qual sua opinião sobre o código abaixo?

```
// format matched kk:mm:ss EEE, MMM dd, yyyy
Pattern timeMatcher = Pattern.compile(
  "\\d*:\\d*:\\d* \\w*, \\w* \\d*, \\d*");
```

- Tem como melhorar?

Comentários relevantes

- Qual sua opinião sobre o código abaixo?

```
// format matched kk:mm:ss EEE, MMM dd, yyyy
Pattern timeMatcher = Pattern.compile(
  "\\d*:\\d*:\\d* \\w*, \\w* \\d*, \\d*");
```

- Tem como melhorar?

- Mover código para uma classe que trata de datas e tempo – elimina o comentário

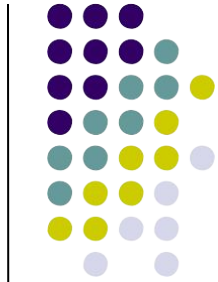
Comentários redundantes

```
/** The name. */
private String name;

/** The version. */
private String version;

/** The licenceName. */
private String licenceName;

/** The version. */
private String info;
```

Comentários redundantes

```
/** The day of the month. */
    private int dayOfMonth;
```

Comentários redundantes

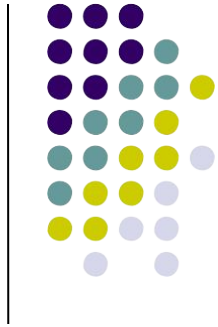


```
/**
 * Default constructor.
 */
protected AnnualDateRule() {
}
```

Comentários redundantes

```
/**
  * Returns the day of the month.
  *
  * @return the day of the month.
  */
 public int getDayOfMonth() {
   return dayOfMonth;
}
```

Comentários redundantes



```
public abstract class ContainerBase
  implements Container, Lifecycle, Pipeline,
  MBeanRegistration, Serializable {

  /**
   * The processor delay for this component.
   */
  protected int backgroundProcessorDelay = -1;


  /**
   * The lifecycle event support for this component.
   */
  protected LifecycleSupport lifecycle =
    new LifecycleSupport(this);


  /**
   * The container event listeners for this Container.
   */
  protected ArrayList listeners = new ArrayList();


  /**
   * The Loader implementation with which this Container is
   * associated.
   */
  protected Loader loader = null;
```

Comentários desatualizados

```
MockRequest request;
private final String HTTP_DATE_REGEXP =
  "[SMTWF][a-z]{2}\\,\\s[0-9]{2}\\s[JFMASOND][a-z]{2}\\s"+
  "[0-9]{4}\\s[0-9]{2}\\:[0-9]{2}\\:[0-9]{2}\\sGMT";
private Response response;
private FitNesseContext context;
private FileResponder responder;
private Locale saveLocale;
// Example: "Tue, 02 Apr 2003 22:18:49 GMT"
```

Código Comentado

```
   InputStreamResponse response = new InputStreamResponse();
   response.setBody(formatter.getResultStream(), formatter.getByteCount());
// InputStream resultsStream = formatter.getResultStream();
// StreamReader reader = new StreamReader(resultsStream);
// response.setContent(reader.read(formatter.getByteCount()));
```

Código Comentado

```
while (!stack.isEmpty()) {

        State current = stack.pop();

//          int theDepth = current.getDepth();
//
//          if(theDepth> maxDepthSearched)
//              maxDepthSearched = theDepth;


        if (current.isGoal(maze)){
            // TODO update the maze if a solution found
            //while(current != null){
            while(current.getParent() != null){
                if(!current.isGoal(maze))
                    maze.setOneSquare(current.getSquare(), '.');
                current = current.getParent();

            }
            return true;
        }
```

Commented-out code: Código "removido" através de comentário

Desvantagens de código comentado

- Código comentado é uma distração e pode confundir o programador

- Código comentado sempre levanta mais questões do que respostas

- Código comentado torna-se defasado ao longo do tempo uma vez que o sistema continua a evoluir enquanto o trecho de código comentado permanece estático

Desvantagens de código comentado

- Código comentado é uma forma errada de controle de versão, uma vez que ferramentas como Git e SVN são indiscutivelmente a solução apropriada

- O simples fato do programador buscar entender a razão por trás do código comentado pode tomar muito tempo

- Programadores esquecerão rapidamente o quão relevante é o código comentado